Project Report for Natural Language Processing

“Equations Knowledge Graph”

Submitted by

Keval Dineshbhai Viradiya

Supervised by

Prof. Dr. Patrick Levi

Ostbayerische Technische Hochschule Amberg Weiden
Department of Electrical Engineering, Media and Computer Science

June 25, 2026

1 Methodology

1.1 System Architecture

The pipeline has seven phases and uses only arXiv sources. HTML is tried first because MathML usually preserves clean LaTeX and nearby prose; PDF is used when HTML is missing or incomplete. This follows MathML aware LaTeX to HTML conversion work [1].

One important choice is to keep three things separate: the order of the equations, the equation text, and the surrounding context. HTML gives cleaner LaTeX but can lose the order, while PDF keeps the order but is noisy; keeping the two separate means the pipeline does not link a symbol or meaning to the wrong equation. The final output is saved as one JSON file.

1.2 Equation and Symbol Extraction

From each paper we take only the numbered equations the author has tagged, and always keep the first seven, so every paper gives a fixed and comparable slice. We try the HTML version first. HTML stores the original LaTeX inside MathML tags, so the equation comes out exactly as the author wrote it. It is the only source that gives the true LaTeX, so we prefer it [1]. We turn to the PDF only when the HTML is missing or its equation order is broken, since a PDF does not keep the math structure. To reduce errors in that case, we run three engines in parallel, PyMuPDF, `pdftohtml`, and `pdfminer`, instead of trusting one. When at least two agree on the order, we merge them and keep the best text per number by a quality score. If the text is still broken, a local OCR tool, Nougat, rebuilds it [2]. Docling only cleans the surrounding prose and never touches the equation LaTeX [3], so every equation is stored in the same LaTeX form, whatever its source.

For the symbols, the HTML path uses MathML `<mi>` tags, which already separate the identifiers from operators and numbers [4]. This keeps standard operators out of the symbol list on its own. For the PDF path, a LaTeX parser collects the candidate symbols, Greek letters are written out as names, and summation indices are removed, since they carry no physical meaning.

1.3 Symbol Definition Extraction

Once the symbols are known, each one needs a short definition from the paper. Our first method was a single where clause regex. It was precise but covered only about 35% of symbols, because many authors do not write "where X is. . . "; they use an appositive phrase or a short verb sentence instead. To raise coverage, a second method added a fixed list of common symbols and used it whenever the regex found nothing. This filled more fields, but it was context blind: H always became "Hamiltonian" and ρ always "density matrix", even when the paper used them differently. Table 1 compares the three methods.

Approach	Coverage	Problem
Single stage where clause regex only	$\sim$35% fill rate	Misses symbols defined as appositives or verb sentences
Single stage regex with predefined symbol fallback	High fill rate	Falls back to a fixed list and returns context blind definitions when regex finds nothing
5 stage document only pipeline (ours)	59.2% (1313/2219 keys)	Honest empty for the rest

Table 1: Comparison of symbol definition extraction approaches.

To keep coverage without these wrong guesses, and following Stathopoulos et al. [5] and related work [6], we replaced the fixed list with a staged pipeline. The stages run in order and stop at the first match. Stage 1 reads where clause patterns, Stage 2 scans for definitor verbs such as "ρ represents. . . ", and Stage 3 uses spaCy noun chunk parsing to catch appositive phrases. When the text gives nothing, Stage 4 falls back to the LaTeX structure: Pauli, gamma, and Lindblad operators have clear shapes the system reads directly, but a shape is accepted only when the equation or prose confirms it, so nothing is labelled blindly. Stage 5 leaves the symbol empty. Unlike the text only work in [5], [6], this structure stage and a later paper wide pass recover symbols defined far from their equation.

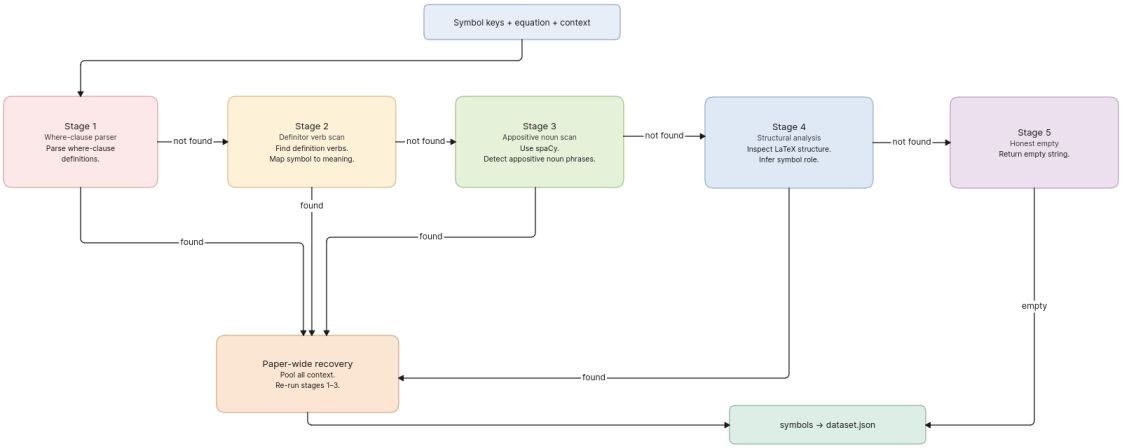

Figure 1.2: Symbol definition extraction pipeline.

A later paper wide pass pools the context of all seven equations and reruns Stages 1 to 3, catching symbols used far from where they are defined. The remaining 40.8% empty rate is honest: those symbols are never defined in the text we read.

1.4 Equation Meaning Extraction

Giving each equation a short name is the hardest task, because the meaning is rarely written in one fixed pattern. A regex of fixed phrases would always miss the forms it was not built for, so we use spaCy dependency parsing instead [7]. The parser reads the grammar of each sentence, not its surface words: when it finds a cue verb such as *define*, *describe*, or *express*, it takes the direct object of that verb as the meaning. This works across sentence types with no hand made list. The stages are tried in order of reliability, from a local cue down to the left hand symbol's name. Before parsing, a noise scrubber removes the inline LaTeX fragments the ar5iv converter leaves between words, since these break the parse tree and caused most errors.

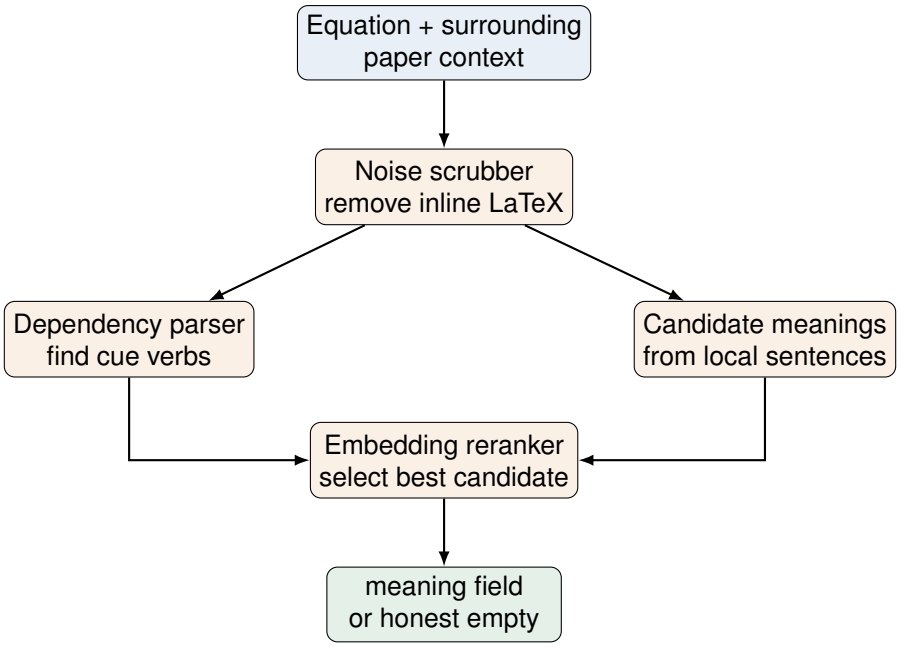

Figure 1.4: Equation meaning extraction pipeline.

When two stages return candidates of equal rank, a sentence embedding reranker breaks the tie. It encodes each candidate and the context with the local `BAAI/bge-base-en-v1.5` model from C-Pack [8], and keeps the one closest to the context; it never writes text, only chooses. If no stage finds evidence, the field is left empty instead of a wrong guess. The resulting 16.3% empty rate is honest: those equations have no name in the paper.

1.5 Relation Detection

Each equation must show a relation to every other equation in the same paper, following the idea of building scientific knowledge graphs from entities and relations [9]. The grade is one of three: `none`, `potential`, or `strong`. The first method linked two equations if they shared common words nearby. This gave too many wrong results, because common physics words like "state" or "field" appear near almost every equation.

The final method combines four signals into a weighted score (Figure 5): shared symbol overlap (same variables), operator type match (similar structure, e.g. both commutators), LHS to RHS cross reference (one equation's result used in another), and section proximity (same section). Each signal has its own score between 0 and 1: a direct LHS to RHS reference scores 0.9 and gives a strong grade, while weak word overlap (about 0.4 to 0.5) gives only potential. If no signal fires, the grade is `none`, so the thresholds depend on the evidence type, not on one single cutoff.

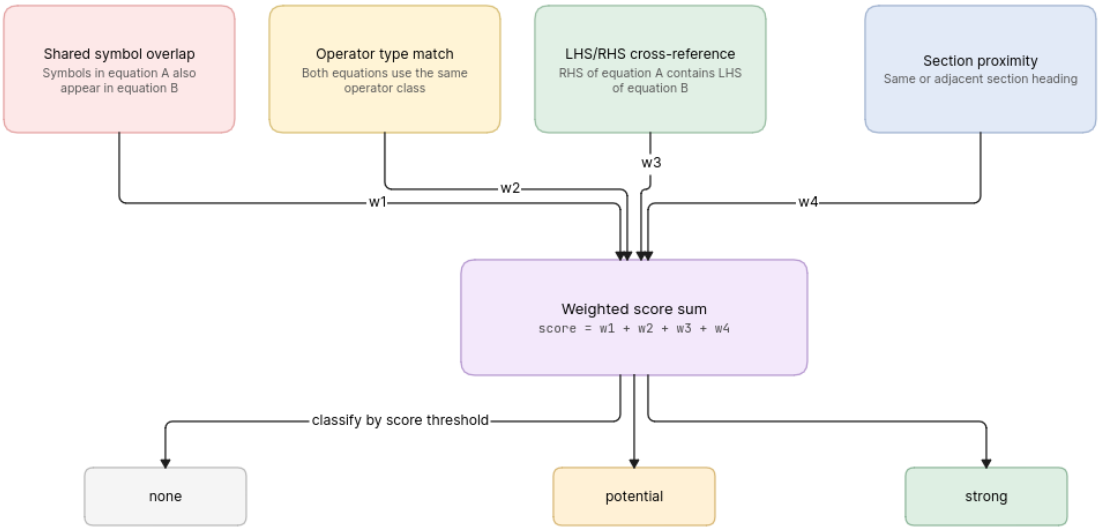

Figure 5: Four signal relation detection pipeline.

1.6 Audit Trail

Each equation carries an audit trail dictionary. Each key is the name of the pipeline function that produced a field, and its value is a short, human readable note on what that function found or decided the sentence a meaning came from, the line a symbol matched, or the signal that set a grade. Because every value points back to real text, any output can be checked against the original paper without running it again.

2 Generalization and Results

Table 2 shows the final dataset statistics for 60 papers and 350 equations. Six papers had no numbered equations, so they count only towards source coverage, not equation records.

Metric	Count	Percentage
Source coverage		
Total papers processed	60	not applicable
HTML source	46	76.7%
PDF source	6	10.0%
Hybrid source	2	3.3%
Papers with no extractable equations	6	10.0%
Equation meaning		
Total equations	350	not applicable
Meaning filled	293	83.7%
Meaning empty (honest)	57	16.3%
Symbol definitions		
Total symbols	2219	not applicable
Symbols with definition	1313	59.2%
Symbols empty (no evidence)	906	40.8%

Table 2: Final dataset statistics.

The pipeline generalizes in three ways. First, it handles any format HTML, PDF, or hybrid with no setting changes. Second, it relies on general patterns rather than fixed lists, so it works across all 60 papers. Third, it stays honest on new papers by leaving a field empty instead of guessing. How well each field turns out then depends on the paper itself: where a meaning, symbol, or link is stated clearly, the pipeline reads it reliably, and the audit trail ties each value back to the sentence it came from.

2.1 Limitations

Symbol definition was the hardest task: many authors use single or Greek letters without explaining them, and no method can find what is never written. Meaning extraction had the same problem: the first method labelled every equation, so many labels were wrong, and leaving the field empty (16.3%) was better than a wrong guess. PDF text also loses sentence breaks and mixes symbols with prose, lowering both fill rates for PDF only papers.

3 AI Tool Use Declaration

AI coding assistants were used in this project to write and debug code, review pipeline logic, and structure the report. AI was also used to help with phrasing and grammar in the writing.

References

[1] H. Stamerjohanns, D. Ginev, C. David, D. Misev, V. Zamdzhiev, and M. Kohlhase, "MathML-aware article conversion from LaTeX: A demonstration of CICM's online LaTeX to XHTML+MathML converter," in *Towards a Digital Mathematics Library*, Masaryk University Press, 2010, pp. 109–120.

[2] L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic, "Nougat: Neural optical understanding for academic documents," *arXiv preprint arXiv:2308.13418*, 2023. arXiv: 2308.13418 [cs.LG].

[3] C. Auer et al., "Docling technical report," *arXiv preprint arXiv:2408.09869*, 2024. arXiv: 2408.09869 [cs.CL].

[4] M. Schubotz et al., "Semantification of identifiers in mathematics for better math information retrieval," in *Proceedings of the 39th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, ACM, 2016, pp. 135–144. DOI: 10.1145/2911451.2911503

[5] Y. Stathopoulos, S. Baker, M. Rei, and M. Stevenson, "Variable typing: Assigning meaning to variables in mathematical text," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, Association for Computational Linguistics, 2018, pp. 303–312. DOI: 10.18653/v1/N18-1028

[6] G. Y. Kristianto, G. Topić, and A. Aizawa, "Extracting definitions of mathematical expressions in scientific papers," in *Proceedings of the 26th Pacific Asia Conference on Language, Information and Computation (PACLIC)*, 2012, pp. 521–530.

[7] M. Honnibal and I. Montani, *spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing*, https://spacy.io, Explosion AI, 2017.

[8] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff, "C-Pack: Packaged resources to advance general chinese embedding," *arXiv preprint arXiv:2309.07597*, 2023. arXiv: 2309.07597 [cs.CL].

[9] Y. Luan, L. He, M. Ostendorf, and H. Hajishirzi, "Multi-task identification of entities, relations, and coreference for scientific knowledge graph construction," in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, Association for Computational Linguistics, 2018, pp. 3219–3232. DOI: 10.18653/v1/D18-1360